

Algoritmica Grafurilor - Cursul 11

18 dicembre 2020

- 1 Reduceri în timp polinomial pentru probleme pe grafuri
 - Probleme Hamiltoniene
 - Problema comis-voiajorului
- 2 Abordări ale problemelor NP-hard
 - TSP metrică: Algoritmul lui Christofides
 - Colorarea nodurilor: Algoritmul de colorare greedy
- 3 Exerciții pentru seminarul din următoarea săptămână
- 4 Exerciții rezolvate (parțial)

Teorema 1

(Karp, 1972) $SM \leq_P CH$.

Demonstrație. Fie $G = (V, E)$ și $j \in \mathbb{N}$ o instanță a problemei SM. Vom construi în timp polinomial (relativ la $n = |V|$) un graf H astfel încât există o mulțime stabilă S în G cu $|S| \geq j$ dacă și numai dacă H este Hamiltonian.

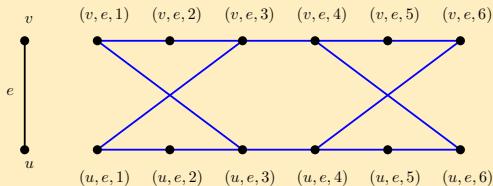
Fie $k = |V| - j$. Să presupunem că $k > 0$ (pentru a evita cazurile banale)

- (i) Fie $A = \{a_1, a_2, \dots, a_k\}$ o mulțime de k de noduri distincte.
- (ii) Pentru fiecare muchie $e = uv \in E(G)$, considerăm graful $G'_e = (V'_e, E'_e)$ cu $V'_e = \{(w, e, i) : w \in \{u, v\}, i = \overline{1, 6}\}$ și $E'_e = \{(w, e, i)(w, e, i+1) : w \in \{u, v\}, i = \overline{1, 5}\} \cup \{(u, e, 1)(v, e, 3), (u, e, 3)(v, e, 1), (u, e, 4)(v, e, 6), (u, e, 6)(v, e, 4)\}$.

Reduceri în timp polinomial - Probleme Hamiltoniene

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Demonstrație (continuare).

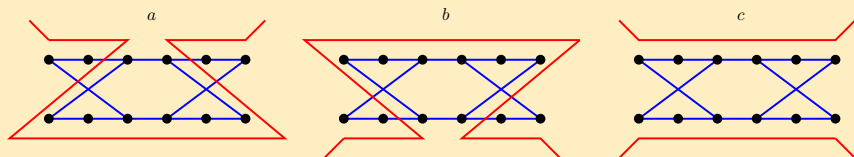


Graful are proprietatea că, dacă este un subgraf indus al unui graf Hamiltonian, H , și nici unul dintre nodurile (w, e, i) cu $w \in \{u, v\}$ și $i = \overline{2, 5}$ nu are alți vecini în H , atunci singurele posibilități de parcurgere a lui G'_e pe un circuit Hamiltonian sunt:

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Reduceri în timp polinomial - Probleme Hamiltoniene

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph



Demonstrație (continuare). Astfel, dacă circuitul Hamiltonian “intră” în G'_e printr-un nod corespunzând lui u ($(u, e, 1)$ sau $(u, e, 6)$) atunci “părăsește” graful G'_e de asemeni printr-un nod corespunzând lui u .

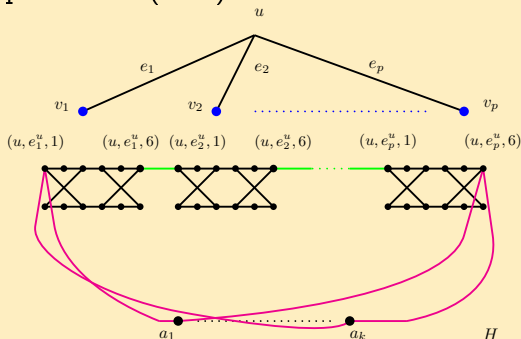
(iii) Pentru fiecare nod $u \in V$, considerăm (într-o ordine arbitrară) muchiile incidente în G cu u : $e_1^u = uv_1, e_2^u = uv_2, \dots, e_p^u = uv_p$ ($p = d_G(u)$). Fie $E_u' = \{(u, e_i^u, 6)(u, e_{i+1}^u, 1) : i = 1, p-1\}$ și $E_u'' = \{a_i(u, e_1^u, 1), a_i(u, e_p^u, 6) : i = 1, k\}$

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Demonstrație (continuare). Graful H are $V(H) = A \cup \left(\bigcup_{e \in E} V'_e \right)$ și

$$E(H) = \left(\bigcup_{e \in E} E'_e \right) \cup \left(\bigcup_{u \in V} (E''_u \cup E'''_u) \right).$$

Evident, poate fi construit din G în timp polinomial (în n).



Demonstrație (continuare). Acum arătăm că există o mulțime stabilă în G cu cel puțin j noduri dacă și numai dacă H este Hamiltonian.

“ $\Leftarrow$ ” Dacă H este Hamiltonian, atunci există C un circuit Hamiltonian în H . Deoarece A este o mulțime stabilă în H , A descompune circuitul C în exact k drumuri intern disjuncte: $D_{a_{i_1} a_{i_2}}, D_{a_{i_2} a_{i_3}}, \dots, D_{a_{i_k} a_{i_1}}$.

Fie $D_{a_{i_j} a_{i_{j+1}}}$ un astfel de drum ($j+1 = 1 + (j \bmod k)$). Din construcția lui H , urmează că primul nod după a_{i_j} pe acest drum va fi $(v_{i_j}, e_1^{v_{i_j}}, 1)$ sau $(v_{i_j}, e_p^{v_{i_j}}, 6)$, unde $p = d_G(v_{i_j})$, $v_{i_j} \in V$.

După aceasta, $D_{a_{i_j} a_{i_{j+1}}}$ va intra în componenta G'_{e_1} sau G'_{e_p} care va fi părăsită printr-un nod corespunzător lui v_{i_j} . Dacă următorul nod nu este $a_{i_{j+1}}$, va intra în componenta corespunzând următoarei muchii incidente cu v_{i_j} , care va fi părăsită de asemeni printr-un nod corespunzător lui v_{i_j} .

Demonstrație (continuare). Urmează că fiecare drum $D_{a_{i_j} a_{i_{j+1}}}$ core-spunde unui singur nod $v_{i_j} \in V$, astfel încât prima și ultima muchie a lui $D_{a_{i_j} a_{i_{j+1}}}$ sunt $a_{i_j}(v_{i_j}, e_t^{v_{i_j}}, x)$, $a_{i_{j+1}}(v_{i_j}, e_{t'}^{v_{i_j}}, x')$, cu $t = 1$ și $t' = d_G(v_{i_j})$, $x = 1$, $x' = 6$ sau $t = d_G(v_{i_j})$, $t' = 1$, $x = 6$, $x' = 1$.

Urmează că nodurile v_{i_j} sunt distincte.

Fie $V^* = \{v_{i_1}, v_{i_2}, \dots, v_{i_k}\}$. Deoarece C este un circuit Hamiltonian în H , urmează că, $\forall e \in E$, există un drum $D_{a_{i_j} a_{i_{j+1}}}$ care traversează G'_e , astfel există $v \in V^*$ incident cu e .

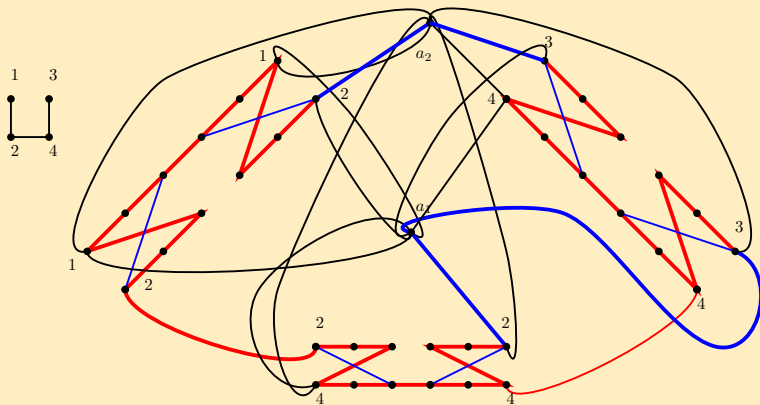
Deci $S = V \setminus V^*$ este o mulțime stabilă în G și $|S| = j$.

Astfel, am arătat că, dacă H este Hamiltonian, atunci în G există o mulțime stabilă cu j noduri și răspunsul la SM pentru instanța G, j este da.

Reduceri în timp polinomial - Probleme Hamiltoniene

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Demonstrație (continuare).



- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Demonstrație (continuare). “ $\Rightarrow$ ” Să presupunem că răspunsul la SM pentru instanța G, j este da, astfel există S_0 o mulțime stabilă în G cu $|S_0| \geq j$. Există $S \subseteq S_0$ cu $|S| = j$. Fie $V^* = V \setminus S = \{v_1, v_2, \dots, v_k\}$. Considerăm în H pentru fiecare $e = uv \in E$:

- cele două drumuri din cazul (c) în G'_e (desenate mai sus) dacă $u, v \in V^*$.
- drumul din cazul (b) în G'_e (desenat mai sus) dacă $u \in V^*$ și $v \notin V^*$.
- drumul din cazul (a) în G'_e (desenat mai sus) dacă $u \notin V^*$ și $v \in V^*$.

La reuniunea tuturor acestor drumuri adăugăm muchiile $a_i(v_i, e_1^{v_i}, 1), (v_i, e_1^{v_i}, 6)(v_i, e_2^{v_i}, 1), \dots, (v_i, e_{p-1}^{v_i}, 6)(v_i, e_p^{v_i}, 1), (v_i, e_p^{v_i}, 6)a_{i+1},$ (cu $p = d_G(v_i)$), pentru $i = \overline{1, k}$.

Ceea ce se obține este un circuit Hamiltonian în H . $\square$

Reduceri în timp polinomial - Problema comis-voiajorului (TSP)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Traveling Salesman Problem - TSP Dat $G = (V, E)$ un graf și $d : E \rightarrow \mathbb{R}_+$ o funcție de pondere nenegativă pe muchiile sale, să se determine un circuit Hamiltonian, H_0 , a. î. suma ponderilor pe muchiile lui H_0 să fie minimă (printre toate circuitele Hamiltoniene ale lui G).

Fie graful G poate fi o rețea constând dintr-o mulțime, V , de orașe împreună cu o mulțime, E , de rute directe între orașe, funcția d oferind, pentru fiecare muchie $uv \in E$, $d(uv) =$ distanța pe ruta directă între orașele u și v . Fixând un oraș de start v_0 , circuitul Hamiltonian H_0 reprezintă cel mai scurt traseu care vizitează toate orașele exact odată (cu excepția lui v_0) pe care îl poate parcurge un comis-voiajor plecând din v_0 și întorcându-se în v_0 .

Aceasta este probabil, cea mai studiată problemă de optimizare NP-hard!

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Reduceri în timp polinomial - Problema comis-voiajorului (TSP)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Considerăm următoarea formulare echivalentă a acestei probleme.

TSP Dat $n \in \mathbb{N}$ ($n \geq 3$) și $d : E(K_n) \rightarrow \mathbb{R}_+$, să se determine un circuit Hamiltonian, H_0 , în K_n , cu $d(H_0)$ minim printre toate circuitele Hamiltoniene ale lui K_n , unde

$$d(H_0) = \sum_{e \in E(H_0)} d(e).$$

Dacă graful G , pe care trebuie să rezolvăm TSP, nu este graful complet K_n , atunci putem introduce muchiile lipsă cu o pondere foarte mare, $M \in \mathbb{R}_+$, unde $M > |V| \cdot \max_{e \in E(G)} d(e)$.

Aceasta este formularea problemei TSP simetrice, o problemă similară (asimetrică) poate fi considerată pentru cazul când G este un digraf. În studiul complexității timp a acestei probleme, vom considera $d(e) \in \mathbb{N}$, pentru fiecare muchie e .

Reduceri în timp polinomial - Problema comis-voiajorului (TSP)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Problema de decizie asociată este

DTSP

Instanță: $n \in \mathbb{N} (n \geq 3)$, $d : E(K_n) \rightarrow \mathbb{N}$ și $B \in \mathbb{N}$.

Întrebare: Există un circuit Hamiltonian, H_0 , în K_n , a. î. $d(H_0) \leq B$?

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Teorema 2

$CH \leq_P DTSP$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

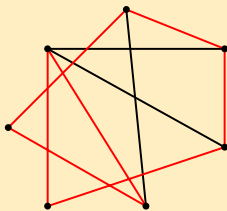
Demonstrație. Fie $G = (V, E)$ ($|V| = n$) o instanță a problemei CH. Construim în timp polinomial o instanță, $d : E(K_n) \rightarrow \mathbb{N}$ și $B \in \mathbb{N}$, a problemei DTSP astfel încât există un circuit Hamiltonian în K_n de pondere totală cel mult B dacă și numai dacă G este graf Hamiltonian.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

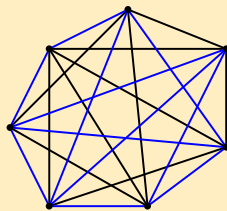
Fie

$$d(vw) = \begin{cases} 1, & \text{dacă } vw \in E(G) \\ 2, & \text{dacă } vw \in E(\overline{G}) \end{cases} \text{ și } B = n.$$

Atunci, există în K_n un circuit Hamiltonian de pondere $\leq n$ dacă și numai dacă există un circuit Hamiltonian C în K_n astfel încât $\forall e \in E$ $d(e) = 1$, adică, dacă și numai dacă G are un circuit Hamiltonian:



G



weight 2
weight 1

Urmează că TSP este o problemă NP-hard. $\square$

Reduceri în timp polinomial - Problema comis-voiajorului (TSP)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

O abordare posibilă este să considerăm un algoritm de aproximare $\mathcal{A}$, care să construiască, în timp polinomial, pentru fiecare instanță TSP, un circuit Hamiltonian $H_{\mathcal{A}}$ al lui K_n , o aproximare a soluției optime H_o . Calitatea aproximării poate fi exprimată folosind următorul raport:

$$R_{\mathcal{A}}(n) = \sup_{d: E(K_n) \rightarrow \mathbb{R}_+, d(H_o) \neq 0} \frac{d(H_{\mathcal{A}})}{d(H_o)}$$

$$R_{\mathcal{A}} = \sup_{n \geq 3} R_{\mathcal{A}}(n).$$

Evident, algoritmul de aproximare $\mathcal{A}$ este util numai dacă $R_{\mathcal{A}}$ este finită. Din nefericire, dacă funcția de pondere d este arbitrară, condiția ca $R_{\mathcal{A}}$ să fie finită este la fel de dificilă ca și rezolvarea exactă a TSP. Mai precis, avem următorul rezultat:

- Graph Algorithms - C. Croitoru - Graph Algorithms - C. Croitoru - Graph Algorithms

Teorema 3

Dacă există un algoritm de aproximare în timp polinomial A pentru TSP a. î. $R_A < \infty$, atunci problema CH poate fi rezolvată în timp polinomial.

Demonstrație. Fie A un algoritm de aproximare în timp polinomial cu $R_A < \infty$. Există $k \in \mathbb{N}$ astfel încât $R_A \leq k$. Fie $G = (V, E)$ un graf arbitrar, instanță a CH. Dacă $n = |V|$, atunci considerăm $d : E(K_n) \rightarrow \mathbb{N}$ definită prin

$$d(uv) = \begin{cases} 1, & \text{dacă } uv \in E(G) \\ kn, & \text{dacă } uv \notin E(G) \end{cases}.$$

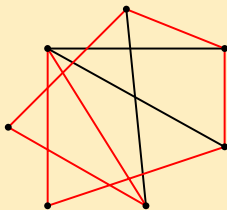
Evident, G este Hamiltonian dacă și numai dacă H_o , soluția optimă a acestei instanțe a TSP, satisface $d(H_o) = n$.

Reduceri în timp polinomial - Problema comis-voiajorului (TSP)

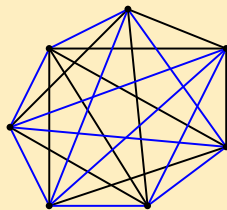
Aplicăm $\mathcal{A}$ pentru a rezolva aproximativ această instanță a TSP.

- Dacă $d(H_{\mathcal{A}}) \leq kn$, atunci $d(H_{\mathcal{A}}) = n$ și $H_{\mathcal{A}} = H_0$.
- Dacă $d(H_{\mathcal{A}}) > kn$, atunci $d(H_0) > n$. Într-adevăr, presupunând că $d(H_0) = n$, avem $\frac{d(H_{\mathcal{A}})}{d(H_0)} \leq k$, astfel $d(H_{\mathcal{A}}) \leq kd(H_0) = kn$, contradicție.

Urmează că G este Hamiltonian dacă și numai dacă $d(H_{\mathcal{A}}) \leq kn$, și, deoarece $\mathcal{A}$ rulează în timp polinomial, urmează că CH poate fi rezolvată în timp polinomial. $\square$



G



weight $7k$
weight 1

Reduceri în timp polinomial - Problema comis-voiajorului (TSP)

Remarcă

Teorema 3 poate fi formulată echivalent astfel:

Teoremă

Dacă $P \neq NP$, atunci nu există algoritm de aproximare în timp polinomial $\mathcal{A}$ cu $R_{\mathcal{A}} < \infty$.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C.
Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Teorema 4

(Christofides, 1976) Dacă în TSP funcția de pondere d satisface

$$\forall u, v, w \in V(K_n) \text{ distincte, } d(uv) \leq d(uw) + d(wv),$$

atunci există un algoritm de aproximare în timp polinomial $\mathcal{A}$ cu $R_{\mathcal{A}} = 3/2$.

Demonstrație. Fie $\mathcal{A}$ următorul algoritm:

- Determină T^0 , mulțimea de muchii a unui arbore parțial de cost minim din K_n (costul unei muchii e va fi $d(e)$) (acest pas ia un timp polinomial folosind orice algoritm pentru MST).
- Determină M^0 , un cuplaj perfect de pondere minimă în subgraful indus în K_n de mulțimea de noduri de grad impar din arborele parțial de cost minim T^0 (acest pas ia un timp polinomial folosind orice algoritm pentru determinarea unui cuplaj de cardinal maxim).

Demonstrație (continuare)

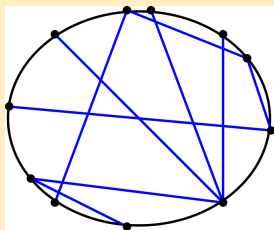
- În multigraful obținut din $\langle T^0 \cup M^0 \rangle_{K_n}$, prin duplicarea muchiilor din $T^0 \cap M^0$ (este conex și are toate nodurile de grad par) determină un parcurs Eulerian închis, $(v_{i_1}, v_{i_2}, \dots, v_{i_1})$. Eliminăm toate aparițiile multiple ale nodurilor interne pentru a obține un circuit Hamiltonian H_A în K_n cu mulțimea de muchii $H_A = \{v_{j_1} v_{j_2}, v_{j_2} v_{j_3}, \dots, v_{j_n} v_{j_1}\}$ (amândouă construcțiile se fac în $\mathcal{O}(n^2)$).

Fie H_A soluția aproximativă a TSP dată de algoritmul lui Christofides. Fie $m = \lfloor n/2 \rfloor$ și H_o o soluție optimă. Arătăm (după Cornuejols & Nemhauser) că

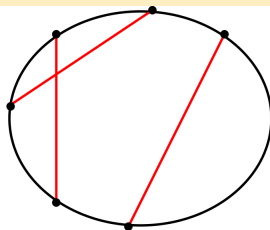
$$\forall n \geq 3, d(H_A) \leq \frac{3m-1}{2m} d(H_o).$$

Abordări ale problemelor NP-hard - Algoritmul lui Christofides

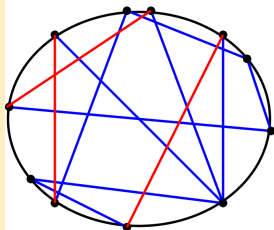
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms



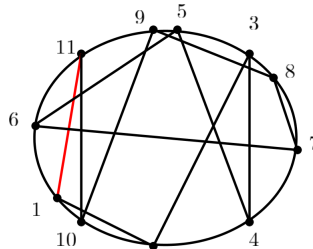
T^0



M^0



Eulerian graph



H_A

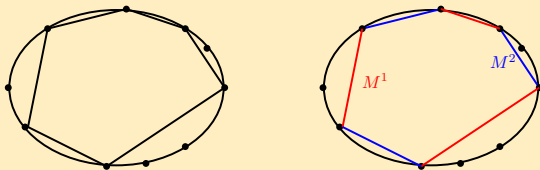
Demonstrație (continuare) Fie $H_o = \{v_1 v_2, v_2 v_3, \dots, v_n v_1\}$ (dacă e necesar, redenumim nodurile).

Fie $W = \{v_{i_1}, v_{i_2}, \dots, v_{i_{2k}}\}$ mulțimea nodurilor de grad impar din $\langle T^0 \rangle_{K_n}$, $i_1 < i_2 < \dots < i_{2k}$. Fie $H = \{v_{i_1} v_{i_2}, v_{i_2} v_{i_3}, \dots, v_{i_{2k-1}} v_{i_{2k}}, v_{i_{2k}} v_{i_1}\}$ circuitul generat de W în K_n . Aplicând în mod repetat inegalitatea triunghiulară, obținem $d(H) \leq d(H_o)$, (ponderea fiecărei corzi, $d(v_{i_j} v_{i_{j+1}})$ este majorată de suma ponderilor de pe muchiile lui H_o care unesc extremitățile corzii $v_{i_j} v_{i_{j+1}}$).

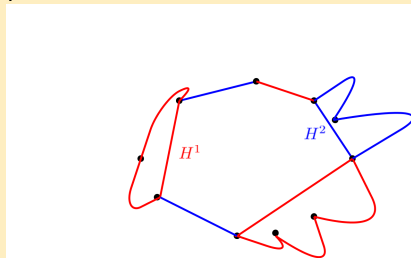
Deoarece H este un circuit par, poate fi scris ca reuniunea a două cuplaje perfecte în $[W]_{K_n}$, $M^1 \cup M^2$. Să presupunem că $d(M^1) \leq d(M^2)$.

Din modul de alegere al lui M^0 , avem $d(M^0) \leq d(M^1) \leq (1/2)[d(M^1) + d(M^2)] = (1/2)d(H) \leq (1/2)d(H_o)$. Fie $\alpha \in \mathbb{R}_+$ a. î. $d(M^0) = \alpha d(H_o)$. Evident, $0 < \alpha \leq 1/2$.

Demonstrație (continuare)



Descompunem H_o în $H^1 \cup H^2$ luând în H^i muchiile din H_o care conectează extremitățile fiecărei corzi din M^i : $(v_{i_j} v_{i_{j+1}} \in M^i \Rightarrow v_{i_j} v_{i_j+1}, \dots, v_{i_{j+1}-1} v_{i_{j+1}} \in H^i)$.



Demonstrație (continuare) Din inegalitatea triunghiulară, $d(H^i) \geq d(M^i)$, $i = 1, 2$. Cel puțin unul dintre H^1 sau H^2 are cel mult $m = \lfloor n/2 \rfloor$ muchii. Să presupunem că H^1 are această proprietate. Deoarece $d(H^1) \geq d(M^1) \geq d(M^0) = \alpha d(H_o)$, urmează că există $e \in H^1$ astfel încât $d(e) \geq (\alpha/m)d(H_o)$.

Fie T un arbore parțial de cost minim obținut din H_o prin ștergerea unei muchii de pondere maximă. Avem $d(T) = d(H_o) - \max_{e \in E(H_o)} d(e) \leq d(H_o) - (\alpha/m)d(H_o)$. Deoarece T^0 este arbore parțial de cost minim în K_n urmează că $d(T^0) \leq d(H_o)(1 - \alpha/m)$.

Folosind inegalitatea triunghiulară obținem

$$\begin{aligned} d(H_A) &\leq d(T^0) + d(M^0) \leq d(H_o) \left(1 - \frac{\alpha}{m}\right) + \alpha d(H_o) = \\ &= \left(1 + \frac{\alpha(m-1)}{m}\right) d(H_o) \stackrel{\alpha \leq 1/2}{\leq} \frac{3m-1}{2m} d(H_o), \forall n \geq 3. \quad \square \end{aligned}$$

Abordări ale problemelor NP-hard - Algoritmul de colorare greedy

Fie $G = (V, E)$ un graf, $V = \{1, 2, \dots, n\}$ și fie π o permutare a lui V .
Construim o colorare a nodurilor $c : V \rightarrow \{1, \dots, \chi(G, \pi)\}$.

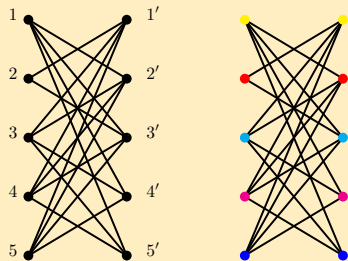
```
 $c(\pi_1) \leftarrow 1; \chi(G, \pi) \leftarrow 1; S_1 \leftarrow \{\pi_1\};$   
for  $(i = 2, n)$  do  
   $j \leftarrow 0;$   
  repeat  
     $j++; v \leftarrow$  primul nod (în  $\pi$ ), din  $S_j$  a. î.  $\pi_i v \in E(G);$   
    if  $(\exists v)$  then  
       $first(\pi_i, j) \leftarrow v;$   
    else  
       $first(\pi_i, j) \leftarrow 0; c(\pi_i) \leftarrow j; S_j \leftarrow S_j \cup \{\pi_i\};$   
    end if  
  until  $(first(\pi_i, j) = 0 \text{ sau } j = \chi(G, \pi))$   
  if  $(first(\pi_i, j) \neq 0)$  then  
     $c(\pi_i) \leftarrow j + 1; S_{j+1} \leftarrow \{\pi_i\}; \chi(G, \pi) \leftarrow j + 1;$   
  end if  
end for
```

Abordări ale problemelor NP-hard - Algoritmul de colorare greedy

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Să observăm că numărul de culori folosite de algoritm nu este mai mare decât $1 + \Delta(G)$. Urmează că $\chi(G) \leq 1 + \Delta(G)$.

$\chi(G, \pi)$ numărul de culori returnate de algoritmul de colorare greedy, poate fi cu oricât mai mare decât $\chi(G)$. De exemplu, fie G graful obținut din graful complet bipartit $K_{n,n}$, cu mulțimea de noduri $\{1, 2, \dots, n\} \cup \{1', 2', \dots, n'\}$, prin ștergerea muchiilor $11', 22', \dots, nn'$.



Abordări ale problemelor NP-hard - Algoritmul de colorare greedy

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Dacă $\pi = (1, 1', 2, 2', \dots, n, n')$, atunci algoritmul de colorare greedy returnează $c(1) = c(1')$, $c(2) = c(2') = 2$, $c(n) = c(n') = n$. Astfel, $\chi(G, \pi) = n$, pe când $\chi(G) = 2$.

Pe de altă parte, pentru orice graf G , există o permutare π a nodurilor sale astfel încât $\chi(G, \pi) = \chi(G)$.

Într-adevăr, fie $S_1, S_2, \dots, S_{\chi(G)}$ clasele de colorare ale unei colorări optime, astfel încât fiecare S_i este o mulțime stabilă maximală (relativ

la incluziune) în $G - \bigcup_{j=1}^{i-1} S_j$. Fie π o permutare care induce o ordonare

astfel încât nodurile apar în ordinea crescătoare a culorilor lor. Atunci, $\chi(G, \pi) = \chi(G)$.

O condiție suficientă pentru corectitudinea algoritmului de colorare greedy:

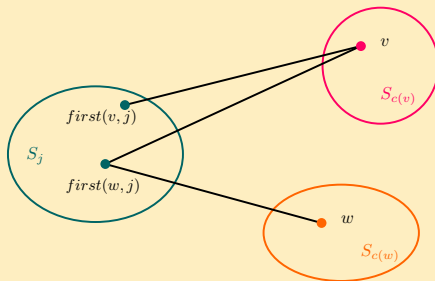
Abordări ale problemelor NP-hard - Algoritmul de colorare greedy

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Teoremă

Dacă, $\forall vw \in E$ și $\forall j < \min\{c(v), c(w)\}$ astfel încât $first(v, j) < first(w, j)$ în ordonarea dată de π , avem $first(w, j)v \in E$, atunci $\chi(G, \pi) = \chi(G)$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru



- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul din următoarea săptămână

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 1. Regiunile (inclusiv cea infinită) formate de n cercuri în plan pot fi colorate cu două culori astfel ca două regiuni care au în comun un arc de cerc (nedegenerat) să fie colorate diferit.

Exercițiul 2. Fie $G = (V, E)$ un graf fără circuite impare disjuncte. Arătați că G este 5-colorabil.

Exercițiul 3. Pentru un graf dat H definim gradul mediu ca $ad(H) = \frac{2|E(H)|}{|V(H)|}$. Pentru un graf G , gradul mediu maxim este

$$mad(G) = \max \{ ad(H) : H \text{ subgraf indus al lui } G \}$$

Fie $k \geq 3$ un întreg. Arătați că, dacă un graf G are numărul cromatic strict mai mare decât k iar gradul său mediu maxim cel mult k , atunci G conține un subgraf indus k -regulat.

Exercițiul 4.

- (a) Arătați că orice graf poate colorat (pe noduri) cu $\Delta(G) + 1$ culori.
- (b) Considerăm următorul algoritm recursiv pentru colorarea nodurilor unui graf 3-colorabil cu n noduri:

```
color( $G$ ) {  
  if ( $\Delta(G) \leq \sqrt{n}$ ) then  
    colorează toate nodurile lui  $G$  cu  $(\Delta(G) + 1)$  culori noi;  
  else  
    fie  $x_0 \in V(G)$  a. î.  $d_G(x_0) = \Delta(G)$ ;  
    colorează  $x_0$  cu o culoare nouă;  
    colorează nodurile lui  $[N_G(x_0)]_G$  cu două culori noi;  
     $G \leftarrow G - (\{x_0\} \cup N_G(x_0))$ ;  
    color( $G$ );  
  end if }
```

Arătați că algoritmul de mai sus folosește $\mathcal{O}(\sqrt{n})$ culori.

Exercițiul 5. Considerăm următoarea problemă:

Set Cover

Instanță: O mulțime nevidă X , o familie de submulțimi ale lui X : $\mathcal{F} = \{X_1, \dots, X_n\}$ și $k \in \mathbb{N}^*$.

Întrebare: X poate fi acoperit cu cel mult k submulțimi din $\mathcal{F}$?

Considerăm următoarea heuristică (de tip greedy) pentru rezolvarea acestei probleme

```
 $\mathcal{F}' \leftarrow \emptyset;$   
while  $(\exists x \in X \text{ neacoperit de } \mathcal{F}')$  do  
    fie  $X_i$  din  $\mathcal{F}$  cu un număr maxim de elemente neacoperite;  
     $\mathcal{F}' \leftarrow \mathcal{F}' \cup \{X_i\};$   
end while  
return  $\mathcal{F}'$ ;
```

Arătați că dacă m este cardinalul unei subfamilii optime a lui $\mathcal{F}$, atunci algoritmul de mai sus returnează o subfamilie cu cel mult $m \ln n$ submulțimi ($n = |X|$).

Exercițiul 6. Fie $G = (V, E)$ un graf cu n noduri și m muchii. O ordonare $\{x_1, x_2, \dots, x_n\}$ a nodurilor lui G este k -mărginită dacă în digraful $\vec{G}$, obținut din G prin înlocuirea fiecărei muchii $x_i x_j$ cu arcul $x_{\min\{i,j\}}, x_{\max\{i,j\}}$, avem $d_{\vec{G}}^+(x) \leq k, \forall x \in V$.

- (a) Descrieți un algoritm care să testeze în $\mathcal{O}(n + m)$ dacă G , are o ordonare k -mărginită, pentru un $k \in \mathbb{N}^*$.
- (b) Folosiți algoritmul de mai sus pentru a determina în $\mathcal{O}((n + m) \log n)$ numărul

$$o(G) = \min\{k \in \mathbb{N} : G \text{ are o ordonare } k\text{-mărginită.}\}$$

- (c) Arătați că orice graf G are o colorare a nodurilor cu $o(G) + 1$ culori.

Exerciții pentru seminarul din următoarea săptămână

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 7. Un algoritm greedy pentru colorarea nodurilor unui graf $G = (V, E)$ este următorul: mai întâi, alege o D -ordonare a nodurilor $V = \{v_1, v_2, \dots, v_n\}$, i. e., $d_G(v_1) \geq d_G(v_2) \geq \dots \geq d_G(v_n)$, apoi v_i primește cea mai mică culoare nefolosită de vecinii săi deja colorați. Considerăm următoarea problemă de decizie

3GCOLOR

instanță: $G = (V, E)$ un graf.

întrebare: Are G o D -ordonare a. î. euristica de mai sus folosește cel mult 3 culori?

Arătați că $3\text{COLOR} \leq_P 3\text{GCOLOR}$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 8. Fie $G = (V; E)$ un graf cu $V = \{1, 2, \dots, n\}$ și $\omega(G) = 2$. Definim un graf nou $M(G)$ considerând reuniune disjunctă a lui G cu $K_{1,n}$ (a cărei bipartiție este $(\{0\}, \{1', 2', \dots, n'\})$) și adăugând toate muchiile $\{i'j, ij' : ij \in E(G)\}$.

- (a) Arătați că $\omega(M(G)) = 2$ și $\chi(M(G)) = \chi(G) + 1$.
- (b) Arătați că, pentru orice $p \in \mathbb{N}^*$, există un graf K_3 -free cu numărul cromatic p .

Exercițiul 9. Fie S o societate formată din n indivizi. Fiecare individ, $i \in S$, cunoaște o submulțime $c(i) \subseteq S \setminus \{i\}$ formată din alți indivizi. Doi indivizi diferiți $i, i' \in S$ nu pot face parte din același juriu dacă unul dintre ei îl cunoaște pe celălalt (putem avea jurii formate dintr-un singur membru).

Arătați că dacă fiecare individ cunoaște cel mult k alți indivizi ($|c(i)| \leq k, \forall i \in S$), atunci există o familie de cel mult $(2k + 1)$ jurii disjuncte care acoperă întreaga societate.

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 1. Soluție.

- Inducție după n :
- Pentru $n = 1$ avem doar două regiuni, să presupunem ca proprietatea este adevărată pentru $n - 1$ cercuri și alegem n cercuri în plan.
- Dacă C este unul dintre aceste cercuri atunci colorăm mai întâi (cu două culori) regiunile obținute prin intersecția celorlalte $n - 1$ cercuri.
- În ceea ce privește cercul C : culorile regiunilor din afara lui C pot rămâne aceleași în timp ce culorile regiunilor interioare lui C se schimbă (de ce?).

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 2. Soluție.

- Dacă G nu conține circuite impare, $\chi(G) \leq 2$ (de ce?).
- Altfel, fie C un circuit impar indus, $\chi(C) = 3$; $G - C$ este bipartit (de ce?).
- Să presupunem că mulțimea nodurilor sale poate fi partiționată:
 $V \setminus V(C) = S_1 \dot{\cup} S_2$, unde S_1 și S_2 sunt stabile.
- Există o 3-colorare a lui C (de ce?), c' iar $c : V \rightarrow \{1, 2, 3, 4, 5\}$, unde

$$c(x) = \begin{cases} c'(x), & x \in V(C) \\ 4, & x \in S_1 \\ 5, & x \in S_2 \end{cases},$$

este o 5-colorare a lui G (de ce?).

Exercițiul 3. Soluție.

- Demonstrăm mai întâi că, dacă un subgraf indus H al lui G are $\delta(H) \geq k$, atunci H este un graf k -regulat:

$$k \geq mad(G) \geq ad(H) = \frac{1}{|H|} \sum_{x \in V(H)} d_H(x) \geq$$

$$\geq \frac{1}{|H|} \sum_{x \in V(H)} \delta(H) = \delta(H) \geq k. \text{ (de ce?)}$$

- Să presupunem prin reducere la absurd că pentru orice subgraf indus H al lui G avem $\delta(H) < k$. Ordonăm nodurile lui G astfel:

Exerciții rezolvate (parțial)

```
 $i \leftarrow n;$   
 $H \leftarrow G;$   
while( $|H| \geq 1$ ){  
  let  $x \in V(H)$  s.t.  $d_H(x) = \delta(H)$ ; //  $d_H(x) < k$   
   $v_i \leftarrow x;$   
   $i \leftarrow i - 1;$   
   $H \leftarrow H - x;$   
}
```

- Evident, $|N_G(v_i) \cap \{v_1, v_2, \dots, v_{i-1}\}| \leq k - 1$.
- Astfel, algoritmul greedy, folosind această ordonare, va oferi o k -colorare a nodurilor lui G (de ce?).

Exercițiul 5. Soluție.

- Fie k_j numărul de elemente rămase neacoperite după j iterații ($k_0 = n$).
- Aceste k_j elemente pot fi acoperite de o subfamilie optimă a lui $\mathcal{F}$, $\mathcal{F}_0$, cu $|F_0| = m$ (de ce?).
- Astfel una dintre submulțimile din $\mathcal{F}_0$ are cardinalul cel puțin k_j/m (de ce?), de unde se obține $k_{j+1} \leq k_j - k_j/m = k_j(1 - 1/m)$ și

$$k_j \leq k_0(1 - 1/m)^j, \forall j \geq 1.$$

- Dar $(1 - 1/m)^j \leq \exp(-j/m)$, deoarece $1 - x \leq \exp(-x), \forall x \in \mathbb{R}$.
- Astfel, $k_j < n \exp(-j/m)$ și $\exp(-j/m)$ devine 1 (de unde $k_j < 1$) pentru $j = m \ln n$.

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 6. Soluție.

- (a) Reformulăm problema: un graf G are o ordonare k -mărginită dacă și numai dacă îi pute ordona nodurile $V = \{v_{i_1}, v_{i_2}, \dots, v_{i_n}\}$ astfel încât $d_{G - \{v_{i_1}, v_{i_2}, \dots, v_{i_{k-1}}\}}(v_{i_k}) \leq k, \forall 2 \leq k \leq n$. (de ce?)
- (Aceasta deoarece într-o ordonare k -mărginită order primul nod are gradul $\leq k$, iar proprietatea este ereditară - de ce?).
 - Următorul algoritm construiește și întreține mulțimea S a nodurilor de grad $\leq k$.
 - Ștergem nodurile din această mulțime unul câte unul și decremen-tăm gradele vecinilor corespunzători.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

```
 $S \leftarrow \emptyset$ ; // a list which contains all the vertices having degrees  $\leq k$  in the current graf;  
 $i \leftarrow 0$ ;  
for  $v \in V(G)$  { // this loop has  $\mathcal{O}(n + m)$  time complexity;  
     $index[v] \leftarrow 0$ ;  
    find  $d_G(v)$ ;  
     $S[v] \leftarrow 0$ ;  
    if  $d_G(v) \leq k$  {  
        push( $S, v$ );  
         $S[v] \leftarrow 1$ ;  
    }  
}
```

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

```
while( $S \neq \emptyset$ ) { // this loop has  $\mathcal{O}(n + m)$  time complexity;
     $x \leftarrow \text{pop}(S)$ ;
     $i++$ ;
     $\text{index}[x] \leftarrow i$ ;
    for  $y \in \mathcal{A}(x)$  { //  $G \leftarrow G - x$ ;
        if ( $\text{index}[y] == 0$ ) {
             $d_G(y) --$ ;
            if ( $S[y] == 0$  and  $d_G(y) \leq k$ ) {
                push( $S, y$ );
                 $S[x] \leftarrow 1$ ;
            }
        }
    }
}
return ( $i == n$ );
```

- (b) Presupunem că funcția de mai sus se numește $\text{BoundedOrder}(G, k)$ (returnează true dacă și numai dacă G are o ordonare d -mărginită). Următorul algoritm este o căutare binară a lui $o(G)$.

```
if ( $E(G) == \emptyset$ )
    return 0;
 $a \leftarrow ?$ ; // invariant:  $\text{BoundedOrder}(G, a) == \text{false}$ 
 $b \leftarrow ?$ ; // invariant:  $\text{BoundedOrder}(G, b) == \text{true}$ 
while(?)
    if( $\text{BoundedOrder}(G, \lfloor \frac{a+b}{2} \rfloor)$ )
         $b \leftarrow \lfloor \frac{a+b}{2} \rfloor$ ;
    else
         $a \leftarrow \lfloor \frac{a+b}{2} \rfloor$ ;
return  $b$ ;
```

Exerciții rezolvate (parțial)

(c) Fie $V = \{v_{i_1}, v_{i_2}, \dots, v_{i_n}\}$ o odonare $o(G)$ -mărginită a lui G . Construim o $o(G) + 1$ colorare a lui G folosind algoritmul de mai jos (de ce?)

for $k = n$ down to 1

$$c(v_{i_k}) \leftarrow \min(\{1, 2, \dots, o(G) + 1\} \setminus \{c(v_{i_j}) : v_{i_k} v_{i_j} \in E(G), j > k\})$$

- (Cum v_{i_k} are cel mult $o(G)$ vecini cu index mai mare putem alege întotdeauna o culoare din mulțimea de culori de mai sus.)

Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exercițiul 7. Soluție.

- Fie G un graf (an instanță pentru problema 3COL); construim în timp polinomial o instanță H pentru 3GCOL.
- Când $\Delta(G) \in \{0, 1\}$ definim $H = G$; H are o 3-colorare (de ce?) și pentru orice D-ordonare euristica oferă o 3-colorare (de ce?).
- Când $\Delta(G) \geq 2$ definim H așa încât G este un subgraf indus al lui H , $\Delta(G) = \Delta(H)$, și $d_H(x) = \Delta(G)$, $\forall x \in V(G)$, folosind următorul algoritm:

```
 $H \leftarrow G;$   
while ( $\exists x \in V(G), d_H(x) < \Delta(G)$ ) {  
  let  $x$  such that  $d_H(x) < \Delta(G)$ ;  
   $k \leftarrow \Delta(G) - d_H(x)$ ;  
   $V(H) \leftarrow V(H) \cup \{v_1^x, v_2^x, \dots, v_k^x\}$ ; // these are new noduri;  
   $E(H) \leftarrow E(H) \cup \{xv_1^x, xv_2^x, \dots, xv_k^x\}$ ;  
}
```

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Instanța H a fost construită în timpul $\mathcal{O}(|V(G)|^2)$ plecând de la instanța G .
- Rămâne să arătăm că G admite o 3-colorare dacă și numai dacă există o D-ordonare of $V(H)$ astfel încât, plecând de la ea, euristica folosește cel mult 3 culori.
- " $\implies$ " Fie $c : V(G) \rightarrow \{1, 2, 3\}$ a 3-colorare of $V(G)$, and $S_i = c^{-1}(i)$, $i = 1, 2, 3$.
- Definim următoarea D-ordonare: mai întâi sunt nodurile din S_1 , apoi cele din S_2 , apoi cele din S_3 iar la final nodurile pendante adăugate - dacă există astfel de noduri în H .
- Cu această ordonare aplicăm algoritmul greedy de colorare și obținem o colorare c' :

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

- $c'(v) = 1, \forall v \in S_1$;
 - $c'(v) \in \{1, 2\}, \forall v \in S_2$;
 - $c'(v) \in \{1, 2, 3\}, \forall v \in S_3$;
 - $c'(v) \in \{1, 2\}, \forall v \in V(H) \setminus V(G)$;
- c' este o 3-colorare a lui $V(H)$ (de ce?).
 - " $\Leftarrow$ " dacă euristica oferă o 3-colorare a lui $V(H)$, atunci G este un graf 3-colorabil (de ce?).

Exercițiul 8. Soluție.

- (a) Dacă $M(G)$ conține trei noduri x, y, z care induc un K_3 , atunci trebuie ca $\{x, y\} \subseteq V(G)$ sau $\{x, y\} \subseteq V(K_{1,n})$ (de ce?).
- În primul caz $xy \in E(G)$, $z = u'$ și $xu, yu \in E(G)$ ($u \in V(G)$) adică $\omega(G) \geq 3$, în al doilea caz se obține o contradicție similară în $K_{1,n}$. Astfel, $\omega(M(G)) = 2$.
 - Putem presupune fără a restrânge generalitatea că G nu are noduri izolate (de ce?).
 - Fie $c : V(G) \rightarrow \{1, 2, \dots, \chi(G)\}$ o colorare optimă a lui G . $\chi(M(G)) \geq \chi(G)$ și $c' : V(M(G)) \rightarrow \mathbb{N}^*$, definită prin $c'(i') = c'(i) = c(i)$, $\forall i \in V(G)$, $c(0) = \chi(G) + 1$, este o colorare a lui $M(G)$ (de ce?).

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Deci $\chi(M(G)) \leq \chi(G) + 1$.
- Să presupunem prin reducere a absurd că $\chi(M(G)) = \chi(G)$ și fie $c'_0 : V(M(G)) \rightarrow \{1, 2, \dots, \chi(G)\}$ o colorare optimă a lui $M(G)$, $c'_0(0) = p_0$.
- Definim $c_0 : V(G) \rightarrow \{1, 2, \dots, \chi(G)\} \setminus \{p_0\}$ luând

$$c_0(i) = \begin{cases} c'_0(i), & \text{if } c'_0(i) \neq p_0 \\ c'_0(i'), & \text{if } c'_0(i) = p_0 \end{cases}$$

- c_0 este o colorare a lui G (de ce?) - contradicție.
- Astfel, $\chi(M(G)) = \chi(G) + 1$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 9. Soluție.

- Fie $G = (V, E)$ următorul graf:

$$V = S, \forall i, j \in S, ij \in E \text{ if } i \in c(j) \text{ or } j \in c(i).$$

- Numărul de muchii din G este cel mult $k|S|$ (de ce?).
- Astfel, $2k|S| \geq 2|E| = \sum_{i \in S} d_G(i) \geq |S|\delta(G)$ și trebuie să existe un nod $i \in S$ așa încât $d_G(i) \leq 2k$ (de ce?).
- Orice subgraf al lui G are această (de ce?). Următorul algoritm construiește o $(2k + 1)$ -colorare a lui G :

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

```
while( $|G| > 0$ ) {  
    let  $i \in G$  cu  $d_G(i) \leq 2k$  and  $1 \leq h \leq 2k + 1$  a color not used by its  
    already coloured neighbours;  
     $c(i) \leftarrow h$ ;  
}
```

- Clasele de colorare ale lui c sunt juriile cu proprietatea cerută.